Improving Generalization Capabilities of Models Trained on Time Series Data Using Novel Downsampling Methods

Ofir Landau
Malware Lab, Cyber Security Research Center
Department of Industrial Engineering and Management
Ben-Gurion University of the Negev
Beer-Sheva, Israel
landauof@post.bgu.ac.il

Nir Nissim
Malware Lab, Cyber Security Research Center
Department of Industrial Engineering and Management
Ben-Gurion University of the Negev
Beer-Sheva, Israel
nirni@bgu.ac.il

2025 IEEE International Conference on Data Mining (ICDM) | 979-8-3315-9599-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDM65498.2025.00048

***Abstract*—Learning from time series data, particularly univariate time series data (UTSD), is important in various fields. Learning accurately from UTSD is challenging, since the data may be very granular and noisy and thus may contain time points which do not necessarily contribute to, and may even prevent, the ability to attain high generalization capabilities. Downsampling, in which a pared down version of the original UTSD that contains its most valuable parts is preserved, is commonly performed to cope with highly granular or noisy UTSD. However, there is a lack of effective downsampling methods, along with the appropriate evaluation metrics for them. We propose four new UTSD downsampling methods and evaluate their performance on nine commonly used and publicly available datasets. To enable proper evaluation of the downsampling methods, we have proposed three new evaluation metrics. Our evaluation demonstrates that our downsampling methods outperform state-of-the-art (SOTA) methods in two aspects: (1) our methods better preserve the original UTSD; and (2) more importantly, downsampled UTSD provided by our methods allow machine learning (ML) models to attain higher generalization capabilities compared to UTSD that SOTA downsampled. Particularly, our methods obtained better results in almost all the examined datasets (nearly 90%) in comparison to SOTA.**

Keywords*—UTSD; signal processing; downsampling; data preprocessing

I. Introduction

Time series data appears in numerous domains, such as finance, medical research, weather forecasting, ecological forecasting, and malware detection. Unlike tabular data, a time series entity is composed of multiple rows, each representing a specific timestamp. The amount of data in each time series depends on the frequency at which it was sampled. Generally, a higher sampling frequency is preferred, since it provides a more informative representation of the entity's behavior. Time series data can also present challenges such as missing values, data drift, and concept shift [1], which require special attention in data preprocessing and modeling. To address these challenges, an appropriate and accurate representation of the signal is required; this can be achieved by performing high frequency data sampling. However, a high sampling rate raises its own issues. Using all the available information for each signal has consequences for algorithms, increasing their computational cost and execution time. In some cases, more demanding algorithms simply cannot handle a very large amount of data. In addition, noise may be concealed in highly granular data, limiting the ability to learn from the data. To address these issues, the time series data can be downsampled. In downsampling, only a subset of the time points in the UTSD is sampled [2], reducing algorithms' computational requirements and storage and transmission systems' memory and bandwidth requirements. Downsampling also has some disadvantages. The main drawback of time series downsampling is information loss: when discarding some of the data points in an entity some of the information the original data contains may be lost. In this study, we focus on downsampling methods for UTSD that do not change time series data's essence. Such methods can be applied in the preprocessing stage of any data science task (including classification, regression, and clustering). In contrast, while many other types of data compression methods output data objects that encompass most of the original data, the output itself is no longer a signal, but an encoded version of it that cannot be understood without a decoding phase. As a result, the methods' output cannot serve as input to the same algorithms as the original data, and such methods are limited when considering algorithms and tasks that require a time series as input, since these methods rely on actual time series data as opposed to its representation, such an embedding.

In existing methods [3] for downsampling UTSD, a simple calculation is performed over sliding windows of the original signal; the output is a single point for each window that should represent it, according to a naive criterion regarding what should be removed and what should be kept. The main drawback of such methods is that they do not consider the signal itself. Assuming that the signal contains a more informative part and a less informative part for the task at hand, the downsampling method will remove timestamps from both the informative and the less informative parts equally. This strategy is problematic particularly when considering signals with a large portion that could be non-informative for a given task. Further, while multiple downsampling methods have been proposed, evidence is lacking as to which downsampling method is best suited for specific machine learning (ML) related tasks. The evaluation of downsampling methods is hampered by the lack of evaluation metrics in this domain, as in our literature search, we found no metrics for the evaluation of different downsampling methods. Therefore, the general motivation of the methods that will be discussed in this paper is reducing the

2374-8486/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDM65498.2025.00048

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

computational resources and time needed to execute algorithms over high frequency signals while still using signals that are as similar to the original signals as possible, and allowing high generalization capabilities for the learning models that are trained on the downsampled UTSD, as well as suggesting metrics for evaluating downsampling methods. The contributions of this paper are as follows:

- We present four novel methods for UTSD downsampling that preserve the original data better than state-of-the art (SOTA) downsampling methods. Such generic capability is of great importance since it can be used as a preceding step prior to applying UTSD analysis processes.
- We demonstrate that our proposed downsampling methods improve generalization capabilities of ML models trained on UTSD in the tasks of classification and regression compared to models trained on data downsampled by SOTA downsampling methods in almost all the examined datasets (nearly 90%).
- We define several new metrics by which downsampling methods can be evaluated with respect to their ability to preserve the original signal and the generalization capabilities of ML models trained on the downsampled data. Such metrics will enable and facilitate the comparison of downsampling methods and improve understanding of each method's pros and cons.
- Our evaluation on a variety of UTSD datasets and different learning tasks sheds light on our proposed methods' suitability by examining their use in various domains with different learning algorithms, learning tasks, and dataset sizes.

II. Related Work

The easiest approach for signal downsampling (with N data points) is to select every nth sample (which is equivalent to sampling at a lower frequency in the first place). This will output data with a frequency that is lower than the original frequency by a factor of n. Another approach is to divide the data into $M < N$ bins and average each bin, producing an output of size M [4]. Each approach has benefits. For example, using the average enables all of the original data to be used to some degree when creating the output. On the other hand, taking the first sample of each bin guarantees that the output will be composed of data that is in the original input (unlike taking the mean values which may not reflect existing values in the signal). We will refer to the two approaches as first-bin (f. bin) and mean-bin (m. bin) in this paper.

In a more advanced method [2], the process is as follows: First, the original signal is split into M bins. Next, starting with the first bin, a line is drawn between each point in the first bin to every point in the next bin. This process results in a directed acyclic graph. Then each edge in the graph is assigned a weight based on its length, and the longest path in the graph is returned as the output. This method, while being computationally intensive compared to the other approaches mentioned, outputs a signal that is closer to the original one.

Largest triangle algorithms [2] are another downsampling method. This family of algorithms choose data points based on their "effective areas," which are the areas of triangles between a point and its neighbors. The first version of the largest triangle algorithms splits the signal into M bins and returns a signal that contains the point with the largest effective area in each bin. The next version of the algorithm does something similar, but instead of calculating the effective areas, it calculates the area of the chosen point from the previous bin, the examined point in the current bin, and the average value of the next bin. The last version in this family of algorithms is a dynamic one and therefore yields an unevenly sampled output.

Another method that attempts uneven downsampling focuses on improving dynamic time warping (DTW) [5]. The evaluation of this interesting approach performed was limited: a one-nearest neighbor model was used to measure accuracy on a single dataset.

Our literature review revealed several gaps in the downsampling method domain. The first gap identified is the general lack of studies on time series downsampling. All but the first two relatively simple methods (first-bin and mean-bin) presented in this section were proposed for the context of UTSD visualizations alone, while our study focuses on other tasks such as classification and regression of UTSD.

We also found that a small number of the studies propose downsampling methods that output an unevenly sampled output. It is generally advantageous to sample the signal evenly, since we do not know which parts of the signal contain valuable information for a given task. However, sampling the signal unevenly is a direction worth investigating, since it retains the "full" signal, allowing the identification of the valuable parts before downsampling, which will yield more points from the interesting part in the output. Therefore, we suggest signal downsampling methods that output an unevenly sampled output.

Additionally, to the best of our knowledge, no study has dealt with signal downsampling in the context of learning tasks. This is problematic, because even when one knows which method is best for downsampling a specific signal according to a given criterion (such that the essence of the data is preserved), it is not clear whether the resulting downsampled signal will work best for other learning tasks (for example, tasks in which downsampling is a tool and not the main focus) like classification or regression.

Another gap we identified in this domain is the lack of objective and measurable metrics. All methods presented here were either evaluated using a qualitative survey [2] or were not evaluated at all. Therefore, there is a need for appropriate metrics for evaluating time series data downsampling methods, particularly in the context of data science tasks.

III. Research Questions

1. For which learning tasks (binary classification, prediction, categorization, next timepoint prediction) do the proposed downsampling methods improve the ML models' (i.e., models trained on the data downsampled using the proposed methods) generalization capabilities compared to SOTA downsampling methods?
2. Compared to SOTA methods, to what degree do our proposed downsampling methods better preserve the original UTSD?
3. Using one consolidated metric (based on the new metrics proposed in this study), is it possible to

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

evaluate the relationship between the downsampling abilities of the methods and the generalization capabilities obtained by the learning models trained on the downsampled data?

4. How do the proposed downsampling methods compare to the SOTA methods with respect to time complexity?

IV. Data

In this section, we briefly present the nine datasets used to evaluate the examined methods. Each record in each dataset is a singular piece of time series data. An overview of the datasets is presented in Table 1. The dimensions column contains the number of entities by the number of time points each entity has (the entity's length). In order to comprehensively evaluate the methods, we included datasets from different domains (including geology, finance, and medicine), datasets that are characterized by different time series properties (frequency, number of entities, and the entity's length), and datasets that can be used for different learning tasks (binary classification, multi-class classification, prediction, and regression).

TABLE 1. OVERVIEW OF THE DATASETS UTILIZED FOR EVALUATION

Dataset	*Dimensions*	*Sampling Frequency*	*Task*	*Reference*
Earthquakes	461 X 512	hourly	classification	[9]
Gunpoint	200 X 150	Na	classification	[10]
Urbansound8K	5425 X 44100	44100Hz	classification	[11]
ECG5000	5000 X 140	Na	classification	[12]
Orders	1 X 60	daily	prediction	[14]
Bikes (Hour)	1 X 17389	hourly	prediction	[13]
Bikes (Day	1 X 731	daily	prediction	[13]
Stocks	1 X 1223	daily	prediction	[15]
Weather	20 X 29	daily	Next value prediction	[16]

V. Methods

A. Our Proposed Methods for UTSD Downsampling

We focus on methods that output a signal for which the number of points is smaller than the corresponding input; therefore, we do not include information representation methods like encoder-decoder architectures [6] that employ data compression.

1) Iterative Method

The iterative method (Iter.), presented in Algorithm 1, starts with a calculation of the importance score of each point (the maximal slope between a point and its neighbours, which are the point's immediate predecessor and successor) in the signal and then iteratively drops the point with the lowest score and updates its neighbours' scores. This procedure is performed until the output is the desired size. Algorithm 2 presents the criterion for choosing a point, which is relevant to more than a single algorithm.

Algorithm 1: Iterative downsampling

Input: $signal_{in}, keepratio$

Output: downsampled signal $signal_{out}$

1. $diffs \leftarrow [\infty]$
2. **For** $idx \in [1, \dots, length(signal_{in}) - 1]$:
   1. $imprt_{idx} \leftarrow \boldsymbol{calcImportance}(signal_{in}[idx - 1], signal_{in}[idx], signal_{in}[idx + 1])$
   2. $diffs \leftarrow diffs + [imprt_{idx}]$
3. $diffs \leftarrow diffs + [\infty]$
4. $signal_{out} \leftarrow signal_{in}$
5. **While** $\frac{length(signal_{out})}{length(signal_{in})} > keepratio$:
   1. $idx \leftarrow idxmin(diffs)$
   2. $drop \leftarrow signal_{in}[idx]$
   3. $diffs[idx - 1] \leftarrow \boldsymbol{calcImportance}(signal_{in}[idx - 2], signal_{in}[idx - 1], signal_{in}[idx + 1])$
   4. $diffs[idx + 1] \leftarrow \boldsymbol{calcImportance}(signal_{in}[idx - 1], signal_{in}[idx + 1], signal_{in}[idx + 2])$
   5. $signal_{out} \leftarrow signal_{out} - [drop]$
   6. $diffs \leftarrow diffs - diffs[idx]$
6. **Return** $signal_{out}$

Algorithm 2: Calc Importance

Input: $left, midde, right$

Output: importance of $middle$

1. $mls \leftarrow abs(\frac{right.y - left.y}{right.x - left.x} - \frac{middle.y - left.y}{middle.x - left.x})$
2. $rms \leftarrow abs\left(\frac{right.y - middle.y}{right.x - middle.x} - \frac{right.y - left.y}{right.x - left.x}\right)$
3. **Return** $\max(amls, rms)$

2) Recursive Methods

The recursive method (Rec.), which is presented in Algorithm 3, uses the method for importance calculation from the previous method and the top-most important points according to it. This time, the scores are not updated, as they are calculated recursively. A major difference between the methods is in the "neighbours" for the importance calculation. In Rec. we use the edges of the examined sub-signal (and not the direct neighbours as done in Iter.). The recursive step is presented in Algorithm 4.

Algorithm 3: Recursive downsampling

Input: $signal_{in}, keepratio$

Output: downsampled signal $signal_{out}$

1. $importances \leftarrow \boldsymbol{Recursive\ Step}(signal_{in}, \{\})$
2. $keepnum \leftarrow length(signal_{in}) * keepratio$
3. $signal_{out} \leftarrow \{signal_{in}[idx] | importances[idx] \in top\ keepnum\ highest\ values\ in\ importances\}$
4. **Return** $signal_{out}$

Algorithm 4: Recursive step

Input: $signal_{in}, imprtncs$

Output: importance per point $imprtncs$

1. $left_{idx} \leftarrow signal_{in}[0]_{idx}$
2. $right_{idx} \leftarrow signal[length(signal_{in}) - 1]_{idx}$
3. **If** $length(signal_{in}) \leq 2$:
   1. **Return** $imprtncs$

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

4. $middle_{idx} \leftarrow floor(\frac{left_{idx}+right_{idx}}{2})$
5. **If** $length(signal_{in}) = 3$:
   1. $imprtncs[middle_{idx}] \leftarrow$ $\boldsymbol{calcImportance}(signal_{in}[left_{idx}], signal_{in}[middle_{idx}], signal_{in}[right_{idx}])$
   2. **Return** $imprtncs$
6. $imprtncs \leftarrow$ $\boldsymbol{Recursive\ Step}\begin{pmatrix} signal_{in}[left_{idx}:middle_{idx}], \\ imprtncs \end{pmatrix}$
7. $imprtncs \leftarrow \boldsymbol{Recursive\ Step}(signal_{in}\begin{bmatrix} middle_{idx}: \\ right_{idx} \end{bmatrix}, imprtncs)$
8. $imprtncs[middle_{idx}] \leftarrow$ $\boldsymbol{calcImportance}(signal_{in}[left_{idx}], signal_{in}[middle_{idx}], signal_{in}[right_{idx}])$
9. **Return** $imprtncs$

3) Adaptive Recursive Method

The main difference between the adaptive recursive method (A-Rec.) and Rec. is that in this case, after the index to drop is chosen from the output, the importances of the sections that used the chosen index are recalculated using the closest neighbours that are still in the signal. The main idea is to leverage the advantages of Rec. (whose view is more global than that of Iter.) and Iter. (where the scores are updated after dropping a point). The method's pseudocode is provided in Algorithm 5. The recursive step is presented in Algorithm 6.

Algorithm 5: Adaptive recursive downsampling

Input: $signal_{in}, keepratio$
Output: downsampled signal $signal_{out}$

1. $importances \leftarrow$ $\boldsymbol{AdaptiveRecursive\ Step}(signal_{in}, \{\})$
2. $signal_{out} \leftarrow signal_{in}$
3. **While** $\frac{length(signal_{out})}{length(signal_{in})} > keepratio$:
   1. $idx \leftarrow idxmin(importances)$
   2. $drop \leftarrow signal_{in}[idx]$
   3. $recalculate \leftarrow \{(left, mid, right) | left = idx[middle]\ or\ right = idx[middle]\}$
   4. **For** $(left, middle, right) \in recalculate$:
      1. $newleft \leftarrow left$ $\boldsymbol{if}$ $left \neq idx[middle]$ $\boldsymbol{else}$ $\mathbf{max}\{i | i < idx[middle]$ $\boldsymbol{and}$ $i \in signal_{out}\}$
      2. $newright \leftarrow right$ $\boldsymbol{if}$ $right \neq idx[middle]$ $\boldsymbol{else}$ $\mathbf{min}\{i | i > idx[middle]$ $\boldsymbol{and}$ $i \in signal_{out}\}$
      3. $triplet \leftarrow (newleft, middle, newright)$
      4. $diffs[triplet] \leftarrow$ $\boldsymbol{calcImportance}(signal_{in}[triplet[0]], signal_{in}[triplet[1]], signal_{in}[triplet[2]])$
   5. $signal_{out} \leftarrow signal_{out} - [drop]$
   6. $importances \leftarrow importances - importances[idx]$
4. **Return** $signal_{out}$

Algorithm 6: Adaptive Recursive step

Input: $signal_{in}, imprtncs$s
Output: importance per (left, middle, right) tuple $imprtncs$

1. $left_{idx} \leftarrow signal_{in}[0]_{idx}$
2. $right_{idx} \leftarrow signal[length(signal_{in}) - 1]_{idx}$
3. **If** $length(signal_{in}) \leq 2$:
   1. **Return** $imprtncs$
4. $middle_{idx} \leftarrow signal_{in}[\frac{length(signal_{in})}{2}]$
5. **If** $length(signal_{in}) = 3$:
   1. $imprtncs[(left_{idx}, middle_{idx}, right_{idx})] \leftarrow$ $\boldsymbol{calcImportance}(signal_{in}[left_{idx}], signal_{in}[middle_{idx}], signal_{in}[right_{idx}])$
   2. **Return** $imprtncs$
4. $imprtncs \leftarrow$ $\boldsymbol{Recursive\ Step}\begin{pmatrix} signal_{in}[left_{idx}:middle_{idx}], \\ imprtncs \end{pmatrix}$
5. $imprtncs \leftarrow$ $\boldsymbol{Recursive\ Step}\begin{pmatrix} signal_{in}[middle_{idx}:right_{idx}], \\ imprtncs \end{pmatrix}$
6. $imprtncs[(left_{idx}, middle_{idx}, right_{idx})] \leftarrow$ $\boldsymbol{Calculate\ Slope}((signal_{in}[left_{idx}], signal_{in}[middle_{idx}], signal_{in}[right_{idx}])$
7. **Return** $imprtncs$

4) Standard Deviation-Based Method

Unlike the rest of the methods presented, in the standard deviation-based method (StdB.) a decision is made whether to keep or drop a section of the signal rather than examining each point individually. We hope that this method will be more robust to outliers in the input. The algorithm iterates over different, overlapping sections of the signal (which are determined by the method's hyperparameters), and the standard deviation of the differences between different points is calculated. Next, the sections are sorted based on the standard deviation (lowest values are first). Then, the sections are iterated, and each point of the section currently being examined is marked as a "candidate to drop" – if the point does not appear in any section that was already examined, then it can be dropped. Otherwise, we move to the next section. The idea is only to drop points if all the sections they appear in were marked as sections with relatively low importance (standard deviation). This process continues until the desired output size is achieved. The pseudocode for this method is presented in Algorithm 7. Note that in this case there are two hyperparameters that affect the method's output: (1) the size of a section for the standard deviation calculation; and (2) the size of the overlap between two such consecutive sections. These hyperparameters should be examined for each dataset. Small values for both hyperparameters will result in evaluation of each point using less neighbors and therefore is only recommended in cases where the data is not noisy or unstable.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 7: Std-based downsampling

Input: $signal_{in}, subsigsize, overlap, keepratio$

Output: downsampled signal $signal_{out}$

1. $secsize \leftarrow subsigsize - overlap$
2. $subsignals \leftarrow \{signal_{in}[idx: idx + secsize] | idx \in \{0 * secsize, 1 * secsize \dots length(signal_{in}) - secsize\}\}$
3. $stds \leftarrow \{std(sig) | sig \in subsignals\}$
4. $subsiglas \leftarrow sort(subsiglas)$ \\ *based on the values in stds*, min *first*
5. $signal_{out} \leftarrow signal_{in}$
6. **While** $\frac{length(signal_{out})}{length(signal_{in})} > keepratio$:
    1. $lowstdsub \leftarrow subsiglas[0]$ \\ minimal std
    2. $stds \leftarrow stds - [stds[lowstdsub]]$
    3. $subsiglas \leftarrow subsignals - [lowstdsub]$
    4. **For** $idx \in lowstdsub$:
        1. **If** $size\left(\left\{subsig \middle| \begin{matrix} idx \in subsig, \\ subsig \in subsignals \end{matrix}\right\}\right) = 0$:
            1. $signal_{out} \leftarrow signal_{out} - [idx]$
7. **Return** $signal_{out}$

An example of the input and output for the examined methods is provided in Figure 1. The top graph contains the original signal (the input); the next two graphs (where the signal is in red and orange) contain the output of the existing methods examined in this study; and the next four graphs (where the signal is in different shades of blue) contain the output of our methods. In order to highlight the difference in the sampling criteria, for all methods the output is half the size of the input signal. As can be seen, our proposed methods sampled more points from the "complicated" parts of the signal (meaning more informative time points) and discarded points from the beginning of the signal – where the signal is close to linearly decreasing (in the example provided). Both the mean bin and first bin methods sampled the input equally across its entire range, regardless of the values of the original signal, demonstrating their insensitivity to informative time points.

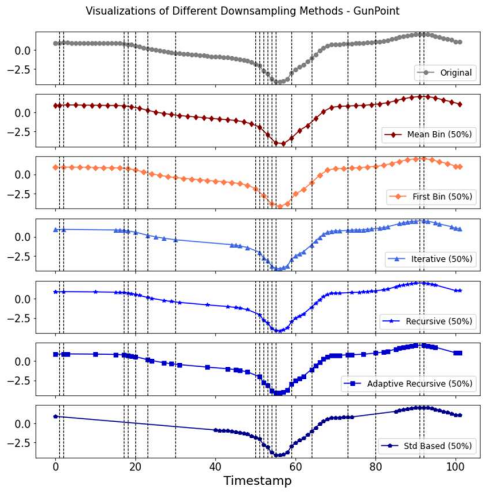


Fig. 1. Example input (gray) and outputs of different downsampling methods (red and orange for SOTA and blue for our proposed methods) when the output size is 50% of the input size.

B. Therotical Analysis

We now present a brief justification for the expected benefit from using our methods in comparison to existing ones. Since it is difficult to present an analysis that fits all scenarios, we will present an analysis that fits cases where we have a dataset of UTSD, that can be easily adapted to a case of a single UTS. Let:

- $D = \{x_1, \dots x_n\}$ be a dataset with n entities, each a UTS
- L a loss function with respect to a task at hand
- ds_{pos} a downsampling method that chooses outputs solely based on the position of current values (such as first-bin)
- ds_{crt} a downsampling method that chooses outputs considering L, using a specific criteria.crt

Our goal is to show that the expected L of using ds_{crt} is no worse than using ds_{pos} under reasonable assumptions. We know that each point in the output of ds_{crt} was chosen to minimize L (because this method is L aware), while ds_{pos} choose output randomly with respect to L (if L was not considering the values themselves, all possible signals would have the same optimal downsampled version – a situation that is not correct in the real world).

Since ds_{crt} performs loss-based selection, and ds_{pos} is not, we can say that:

$$\forall D, L: L\big(ds_{crt}(D)\big) \leq L(ds_{pos}(D))$$

As we desired.

We made several assumptions during this process

1. L is well defined for a task
2. ds_{pos} does not consider such L
3. ds_{crt} considers such L

The second assumption is trivial and was justified in the above analysis. While the other two are task specific, we present a relatively generic view by referring to the information in a time series as information encoded by the data's state, gradient and trend – as it is a common way to refer a time series.

Since L must consider the values of the UTSD, it means it considers the states, gradients or trends of it (or any combination of the three). We can assume that L also maximizes the information presented in the data, as this is the goal in all considered data related tasks (classification, visualization etc.). This satisfies the first assumption to a certain degree (a specific L can be defined based on the task at hand, we merely needed to present the components of L). Now, we need to satisfy the last assumption.

In Iter. method, the criterion aims at keeping points that have the greatest impact on the average state and gradient with respect to the local neighbors of each datapoint. In Rec. and A-Rec. the same calculation is done but with more global neighbors of each point – resulting in more focus on trends rather than gradient (the attention to states is kept). In StdB. The points that have the most impact on local subsets of points in terms of variance are kept – meaning points with special states are kept. In all cases, there is some consideration of either state, gradient or trend in choosing the data points to keep, and by that we satisfy the last assumption. It is important

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

to note that we do not know from this analysis which method will be the best for each case, as it depends on the L defined to a given D.

C. Proposed Downsampling metrics

Although downsampling is very common in signal processing in general and in data science on time series data in particular, there are no metrics to measure the quality of a downsampling method. While comparing the downsampled signal to the original output might provide an adequate measure of how well a method works, such a metric disregards other important aspects regarding the quality of the downsampling method. Such aspects include time and space complexity and, more importantly, the generalization capabilities of an ML model trained on the downsampled signal. In this section, we propose several metrics for evaluating downsampling methods in general and with respect to different learning tasks in particular.

1) Quantile Normalized Difference Index

Given two signals where one (out) is the result of a downsampling procedure of the other (in) and a hyperparameter β, the quantile normalized dissimilarity index (QNDI) of the two is the sum of the distance between the two signals and the relative size of the new signal, normalized by the difference between the 0.9 quantile and the 0.1 quantile. Here, we use a variation of the Euclidian distance for simplicity, but the distance function is configurable and can be chosen based on the given task and data. This sum (QNDI) is also normalized by β, which indicates how much weight to assign to the size of the output compared to the distance between the signals ($|out|, |in|$ denote the size of the output and input). We use the 0.1 and 0.9 quantiles instead of the full range, because this range is sensitive to anomalies in the data (extremely high or low values, which will affect the range but not the quantiles), which will result in assigning a very low weight to the first part of the equation. We assume that most of the signal is within this newly defined "normal range." This metric is presented in (1) and (2).

$$QNDI(in, out, \beta) = (1-\beta) * Dist(in, out) + \beta * \frac{|out|}{|in|} \quad (1)$$

$$Dist(in, out) = \frac{distance(in,out)}{(quantile(0.9,in) - quantile(0.1,in))} \quad (2)$$

2) Performance Difference

In addition to the first metric, which is a "pure" downsampling metric, we present a metric for the evaluation of downsampling in the context of the generalization capabilities obtained in the learning task at hand: the performance difference (PD). Usually, downsampling serves as a preprocessing phase for a learning task (such as classification or regression). The PD is determined by the difference in the generalization capabilities, based on the defined metric of the learning algorithm trained on the data before and after the downsampling process (note that this metric requires evaluation based on another metric), as described in (3). The desirable values of the PD are determined based on the other metric employed; for example, for measures like the MSE, a negative PD is preferred.

$$PD(metric, data, downsampling) = metric(downsampling(data)) - metric(data) \quad (3)$$

3) Performance Difference with Dissimilarity Index

A good downsampling method in the context of learning tasks should output a signal that satisfies the following three attributes: it should be (1) shorter than the original, (2) similar to the original in a way that preserves the essence of the original signal, and (3) informative relative to the learning task. Our last metric is useful when all three attributes are required. The performance difference with quantile normalized dissimilarity index (PDDI) is presented in (4). To avoid undefined expressions (for example, a negative number to the power of half), we only apply the exponent to the absolute value. The hyperparameters α and γ indicate how much weight to give to each part of the metric and β is the hyperparameter presented for QNDI.

$$PPDI(in, out, \alpha, \beta, \gamma) = PD(in, out)^{\alpha} * QNDI(in, out, \beta)^{\gamma} \quad (4)$$

VI. Evaluation and Experimental Design

A. Experiment 1: Comparison of the Methods Based on their Generalization Capabilities

In this experiment, we examine how the examined downsampling methods affect the results of common ML models trained on the methods' output. By evaluating this aspect of the methods' performance on various time series tasks, we hope to see which tasks benefit from the use of our downsampling methods and the existing methods. In this experiment, the datasets presented earlier are used, along with the following models: K-nearest neighbors (KNN), random forest (RF), logistic regression (LR), support vector machine classifier with an RBF kernel (SVM), naive Bayes (NB), and a decision tree (DT). In addition to the downsampling preprocessing, there is another preprocessing step: the calculation of aggregated features – minimum, maximum, standard deviation, and mean value of the series. The metric to optimize depends on the given task: balanced accuracy (BACC) for classification and MSE for regression and prediction. For regression and prediction, common adaptations of the examined models are used (for example, support vector machine regressor instead of the support vector machine classifier with an RBF kernel), and we use the same acronyms for them. We apply the various downsampling methods to each model in order to examine the differences in the values obtained for the relevant metrics for each model and dataset. We chose aggregated features, because we wanted to demonstrate that off-the-shelf learning algorithms can be used with the proposed downsampling methods. Note that these models do not naturally work with time series data and therefore a feature engineering phase is required – aggregated features are a generic option that works well on all datasets [7]. In this experiment, we also examine the correlation between our proposed metrics and existing metrics, in this case, between MSE and the PPDI values, as well as between BACC and the PPDI values. Note that the PPDI requires the results of experiments 1 and 2.

B. Experiment 2: Comparison of the Methods Based on their Preservation of the Original Signal

In this experiment, we compare common downsampling methods to our proposed methods, examining which downsampling method better preserves the meaningful parts of the original data, for a specific output size. It is important to look at the similarity and the output size because they reflect a tradeoff – a signal containing all of the time points of the original signal is probably the best in terms of similarity to the

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

original signal, but this comes at an expense of keeping all of the time points (not downsampling at all) which may prevent or impair to induce a learning model from the data. In contrast, the shortest downsampled signal is an empty signal, which naturally contains no valuable information. For each of the examined downsampling methods, and for each of the datasets, we evaluate the length of the output relatively to the original signal size (50%, 33%, 25%, 20%, 10%). The evaluation of what valuable information the output contains is performed by calculating the distance between the output and input signals; this is done using DTW, as it easily handles signals of different lengths. This is necessary, because each method naturally outputs a signal whose length is shorter than the original target. We also examine the correlation between the value of the PPDI and the mean DTW per dataset and downsampling method, to determine whether the PPDI metric is a valid replacement for the DTW metric in this experiment.

VII. Results

A. Experiment 1: Evaluating Generalization Capabilities

In this experiment, we evaluated the different downsampling methods in the context of the generalization capabilities obtained in several data science-based learning tasks: classification, regression, and next point prediction. The pipeline used in this experiment is relatively simple: we normalized the input using the mean/std scaler, extracted aggregated features, and trained the models. Table 2 presents the method producing the best results for each dataset and output size. The best method is defined as the one yielding the best results across most of the datasets (considering only the best performing model per output size and per dataset); in cases in which there was a tie, we presented all of the best performing methods. As can be seen, our proposed methods outperform the SOTA methods, in almost 90% of the datasets (precisely 87.5% which are 7 out of 8 datasets, as on the ninth dataset, UrbanSound8K, there was a tie between proposed and SOTA method). Moreover, we can also see that the lower the output size, the better the generalization capabilities obtained by sustaining the training ML models using the downsampled signals produced by our methods compared to those produced by the SOTA methods.

TABLE 2. BEST DOWNSAMPLING METHOD PER DATASET PER OUTPUT SIZE.

Dataset	*50%*	*33%*	*25%*	*20%*	*10%*
Earthquakes	StdB.	Iter./StdB.	F. bin/Iter.	Iter./StdB.	Iter.
Gunpoint	Rec/StdB.	Iter.	Iter.	Rec./A-Rec.	Iter.
Urbansound8K	F. bin/StdB.	M. bin	M. bin	F. bin	StdB.
ECG5000	F. bin	M. bin	M. bin	M. bin	M bin
Orders	Rec.	Rec.	Rec.	A-Rec.	A-Rec.
Bikes (Hour)	StdB.	StdB.	StdB.	Rec.	Rec.
Bikes (Day	M. bin	F. bin	Iter.	Iter./A-Rec.	Rec.
Stocks	F. bin	StdB.	Iter.	Rec.	Iter./StdB.
Weather	M. bin	F. bin	StdB.	A-Rec.	Rec.

In Table 3, we present the method producing the best results for each model, where the best method for a model is defined as the one yielding the best results on the model for most of the datasets (again, in case of a tie we presented all the best downsampling methods). We present the results per output size.

TABLE 3. BEST DOWNSAMPLING METHOD FOR EACH MODEL AND OUTPUT SIZE.

Dataset	*50%*	*33%*	*25%*	*20%*	*10%*
DT	StdB.	A-Rec./ F.bin/ M. bin	StdB.	StdB.	M. bin
GNB	Rec.	Rec.	Iter./ F.bin/ M. bin	Iter./ StdB./ M. bin	F. bin
KNN	Rec./ F. bin	StdB./ F. bin	StdB.	A-Rec.	Rec.
LR	StdB.	StdB.	StdB.	Iter./ A-Rec.	Iter./ A-Rec.
RF	A-Rec.	F. bin	StdB.	Iter./ StdB.	M. bin
SVM	StdB.	Iter./ M. bin/ A-Rec	Iter.	M. bin	Iter.

Next, we present the results for each downsampling method on each dataset and output size. For simplicity, we only present the score of the best model for each scenario. At the left-most column of each of the following tables, we also present the score when we did not downsample the original signal data at all. The best result for each dataset and output size is bold.

1) Gunpoint

The results obtained for the GunPoint dataset are presented in Table 4. As can be seen, our methods outperformed the SOTA methods for all examined output sizes. The improvement is between 10% (for output sizes of 0.5 and 0.333) and 1% (for an output size of 0.1).

TABLE 4. GUNPOINT DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE BACC IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	0.84	0.86	0.87	0.83	0.86	0.88
M. Bin		0.86	0.90	0.90	0.89	0.89
Iter.		0.92	**0.98**	**0.94**	0.91	**0.90**
Rec.		**0.96**	0.97	0.93	**0.92**	0.87
A-Rec.		0.95	0.96	0.92	**0.92**	0.86
StdB.		0.95	0.83	0.85	0.86	0.88

2) Earthquakes

The results obtained for the Earthquakes dataset are presented in Table 5. In this case, we can see minor differences among the methods for all output sizes, but we again see that the overall performance of our proposed methods is better – an improvement of 1-4% for all output sizes except 25% where there is a tie.

TABLE 5. EARTHQUAKES DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE BACC IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	0.64	0.63	0.64	**0.65**	0.62	0.62
M. Bin		0.61	0.64	0.62	0.62	0.62
Iter.		0.65	**0.65**	**0.65**	**0.65**	**0.66**
Rec.		0.64	0.64	0.64	0.64	0.63
A-Rec.		0.62	0.63	0.62	0.63	0.63
StdB.		**0.68**	0.65	0.64	**0.65**	0.63

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

3) ECG5000

The results obtained on the ECG5000 dataset are presented in Table 6. On this dataset, the SOTA methods' performance surpassed the results of our proposed methods for all output sizes (perhaps because the data in this dataset has a clear trend), improving the BACC by 2-6%.

TABLE 6. ECG5000 DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE BACC IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	0.56	**0.60**	0.58	0.56	0.54	0.53
M. Bin		0.57	**0.61**	**0.58**	**0.58**	**0.58**
Iter.		0.54	0.55	0.53	0.54	0.52
Rec.		0.55	0.55	0.56	0.53	0.53
A-Rec.		0.55	0.57	0.57	0.56	0.49
StdB.		0.55	0.55	0.58	0.56	0.53

4) UrbanSound8K

The results obtained for the UrbanSound8K dataset are presented in Table 7. This dataset is the most difficult one to learn from; it has the highest number of classes, and it is the largest both in terms of the number of entities and the entities' length (number of time points). The low generalization capabilities obtained by all the learning models stem from the fact that there are 10 classes. A classifier that makes decisions randomly will achieve a 10% BACC, and thus obtaining a BACC value of 30% is considered three times more accurate than a random classifier. We see that the largest difference in the BACC between SOTA methods and ours is 3%. Overall, in this dataset, there is no single best method.

TABLE 7. URBANSOUND8K DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE BACC IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	0.33	**0.33**	0.32	0.32	**0.36**	0.34
M. Bin		0.31	**0.35**	**0.36**	0.33	0.29
Iter.		0.33	0.31	0.30	0.27	0.31
Rec.		0.29	0.32	0.33	0.34	0.30
A-Rec.		0.32	0.30	0.33	0.27	0.27
StdB.		**0.33**	0.32	0.34	0.32	**0.35**

5) Orders

The results for the Orders dataset are presented in Table 8. Here, we see the recursive methods achieve the best MSE for all output sizes.

TABLE 8. ORDERS DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE MSE IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	4156	4233	4141	4170	4220	4172
M. Bin		4213	4129	4215	4125	3298
Iter.		4165	4183	4186	4194	8249
Rec.		**4138**	**3767**	**3121**	4171	3208
A-Rec.		4151	4200	4038	**3683**	**2793**
StdB.		4167	4168	4168	4168	4176

6) Bikes (Day)

The results obtained for the Bikes (Day) dataset are presented in Table 9. As can be seen, the best MSE is achieved with Mean Bin for an output size of 50% (which makes sense, because the larger the output, the more of the original data it contains). However, we can also see that our proposed methods perform better as the output size decreases, and that for an output size of 25% and below, their performance surpasses that of the SOTA methods.

TABLE 9: BIKES (DAY): DOWNSAMPLING METHODS' PERFORMANCE WHEN THE MSE IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	926740	908533	**870095**	1047339	950510	1231270
M. Bin		**818419**	1104656	931999	966733	924434
Iter.		902093	942354	**835346**	**856627**	1165444
Rec.		1055367	1049229	1214324	1119403	**856627**
A-Rec.		1023877	980636	1013447	**856627**	1905290
StdB.		1056513	1056513	1056513	1273056	1167753

7) Bikes (Hour)

The results obtained for the Bikes (Hour) dataset are presented in Table 10. As can be seen, the results achieved are similar to those for the Bikes (Day) dataset, however, the overall best MSE was obtained with an output size of 10%, which is counterintuitive. We believe that this is due to the high frequency of the data (in contrast, the Bikes (Day) dataset is 24 times less dense).

TABLE 10. BIKES (HOUR): DOWNSAMPLING METHODS' PERFORMANCE WHEN THE MSE IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	12672	13426	13487	16156	19919	24100
M. Bin		14470	17296	22630	22321	26315
Iter.		17213	17590	17966	18860	**9871**
Rec.		15722	15639	17805	**17190**	18830
A-Rec.		16300	18154	19251	18860	22036
StdB.		**11030**	**11035**	**11039**	18269	18259

8) Stocks

The results obtained for the Stocks dataset are presented in Table 11. Like the Bikes (Hour) dataset, we see that with the exception of the 50% output size, our proposed methods outperform the SOTA methods.

TABLE 11. STOCKS DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE MSE IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	25905	**22297**	28692	28597	24737	28124
M. Bin		28194	28467	26446	27441	26824
Iter.		24630	28838	25156	**22082**	**20746**
Rec.		27007	24866	**22881**	24726	26755
A-Rec.		24869	24151	27596	27267	**20746**
StdB.		24068	**24068**	24068	24068	21826

9) Weather

The results obtained for the Weather dataset are presented in Table 12. Again, on this dataset, our methods' performance surpasses that of the SOTA methods. It can also be seen that our methods perform relatively better as the output size gets smaller. The best results were obtained for an output size of 10%

TABLE 12. WEATHER DATASET: DOWNSAMPLING METHODS' PERFORMANCE WHEN THE MSE IS OPTIMIZED (BEST MODEL ONLY) FOR EACH OUTPUT SIZE.

Method	*100%*	*50%*	*33%*	*25%*	*20%*	*10%*
F. Bin	230	229	**201**	225	260	253
M. Bin		**216**	210	248	248	232
Iter.		250	227	225	233	197
Rec.		233	255	219	216	**158**
A-Rec.		219	212	248	**205**	197
StdB.		220	212	**212**	212	182

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

Lastly, we present now the results of our analysis of the correlation between the values obtained for the BACC, MSE and PPDI metrics. The correlation between BACC and the value of PPDI is 0.794 while the correlation between the MSE and PPDI values is 0.601; both are considered relatively high correlations [8].

B. Experiment 2: Evaluating Resemblance to Original Signal

In this experiment, we evaluated the downsampling methods by applying them on various datasets and comparing the results in terms of how many time points each downsampling method keeps from the original UTSD and the distance between the output and input signals. We used a window size of 10 when calculating the DTW distance in order to reduce the computation time. The only exception is the UrbanSound8K dataset where we used a window size of 3 and a subset of 1,000 samples from the dataset due to the DTW algorithm's computation time. The results are presented in Table 13 where each row contains the results obtained on one dataset. Due to space limitations, we only present the results for outputs that are 50% of the size of the original (i.e., the downsampling rate); the results obtained were consistent for all examined output sizes (meaning that if a method had a lower DTW than another for an output size 50%, then it also had a lower DTW for output sizes of 10%, 20%, 25%, and 33%).

Of the proposed methods, the Iter. method performed best (yielding the lowest distance between the downsampled UTSD and the original one), and it outperformed all the examined methods on four of the datasets. As can be seen, there is no consensus regarding the best-performing method, as the best methods shared their superiority across the 9 collections and five downsampling rates almost equally. Delving deeper into the results, we also examined the relative improvement between the best performing method of the SOTA methods and our methods. For example, when looking at the weather dataset, we can see that the division between the Rec. method (the best performing of the proposed methods) and the First Bin method (the best performing of the SOTA methods) is 1.145; based on this, we can say that compared to our methods, the SOTA methods improved the results by 14.5%. When performing the same calculation for each dataset, the results demonstrate that in cases where the best performing method was a SOTA method, the SOTA methods outperformed the proposed methods by 5.576, while in cases where the best performing method was one of our proposed methods, the proposed methods outperformed the SOTA methods by 6.550.

TABLE 13. RESULTS (MEAN DTW) OF THE SECOND EXPERIMENT FOR AN OUTPUT SIZE OF 50% (THE OUTPUTS ARE 50% OF THE SIZE OF THE ORIGINAL SIGNAL).

Dataset	SOTA		Proposed			
	F. Bin	***M. Bin***	***Iter.***	***Rec.***	***A-Rec.***	***StdB.***
Earthquakes	98.0	124	10.2	40.0	31.4	55.4
Gunpoint	4.85	5.63	6.01	6.40	6.30	26.3
Urbansound8K	3090	3437	5085	4955	5118	7724
ECG5000	9.74	13.03	11.8	11.2	11.5	22.8
Orders	13.2	22.4	12.2	14.2	14.4	17.7
Bikes (Hour)	2230	2651	4170	3954	2933	3857
Bikes (Day)	78.8	114	64.4	74.8	74.8	89.2
Stocks	201	276	121	166	149	185
Weather	71.8	101	82.5	82.2	83.6	138

After examining several examples of outputs from different datasets, we saw that our proposed methods excel when there are rapid changes in the values and when the data is more challenging, meaning that there is no clear and steady trend in the data. In contrast, the SOTA methods are better suited for data with a clear trend, where dropping data points evenly from different parts of the signal will not change the data's behavior – the trend will be maintained and clear.

As for our proposed metric, the PPDI, we observed a correlation of -0.694 between the values obtained for the PPDI and those obtained for the mean DTW (the correlation was calculated per dataset and output size and then averaged). This is a relatively high correlation [8] in terms of the absolute value, indicating that it can serve as replacement for the DTW. We see that the PPDI metric successfully obtains results that are correlated with the metrics used in experiments 1 and 2, indicating that it can replace them (1) without compromising the results, and (2) while shedding light on downsampling methods' ability to both preserve similarity to the original signal and improve generalization capabilities.

VIII. TIME COMPLEXITY ANALYSIS

In this section, we present the theoretical, worst-case time complexity of each method, assuming that the input is a one-dimensional signal with n timestamps and that the size of the sub-signal for the StdB. method is denoted by m.

First, we can say that the time complexity of calculating a slope at a specific point is $O(1)$, as we perform a constant number of calculations. The time complexity of checking a subsection of the data is $O(n)$, because the worst-case scenario will be checking the entire input (with n points).

For the SOTA methods, we need to perform a single iteration over the signal, therefore the worst-case time complexity is $O(n)$ for both methods.

The Iter. method consists of two main parts: calculating the initial slopes for each timestamp and iteratively removing points and updating their neighbors' slopes. The complexity of the first part is $O(n)$, as we iterate over the input once and perform a constant sized calculation. The second loop, if we consider the algorithm as it is presented in the Methods section, is slower. The complexity of each part in the second loop is $O(1)$ except for finding the index of the point with the minimal slope. Naively, we can find the index by iterating over $diffs$ – a complexity of $O(n)$. This results in a total complexity of $O(n^2)$. However, we can use a min-heap instead of a list for $diffs$. This allows us to find the index of the minimal $diff$ in $O(1)$ time complexity and updating the heap in $O(logn)$ by simply removing the old $diffs$ of the neighbors and inserting their new ones, each of which takes $O(logn)$. This will lead to a total time complexity of $O(n * logn)$ for the second part. However, the first part is now a bit more complex, since it also takes $O(n * logn)$ operations to build the initial heap (n insertions of data points to the heap). Therefore, the total time-complexity using a min-heap is $O(n * logn)$.

The structure of the Rec. method is similar to that of a merge-sort algorithm, and its time complexity is $O(n * logn)$. Dividing a signal into two equally long parts until a signal with less than three points is reached takes at most $O(logn)$ iterations. In each iteration we perform a constant number of calculations, with a total complexity of $O(1)$. Next, we have an unsorted array called $imprtns$. Sorting it for the last part

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.

takes $O(n * logn)$ and using it to get the most important points takes $O(n)$ time. Therefore, the total time complexity is $O(n * logn)$.

The A-Rec. method starts by applying the first part of the Rec. method, which takes $O(n * logn)$. Next, we iteratively remove points and update all of the sub-signals that had that point as one of its edges. In the worst-case scenario, we drop a point that was a part of a signal in each iteration of the first part, meaning that it will be a part of $O(logn)$ sub-signals. The update itself has $O(1)$ complexity, so the inner loop's time complexity is $O(logn)$. Assuming that we can solve the $idxmin$ part using min-heap like we did with the Iter., we can achieve a $O(logn)$ time complexity for the entire loop of the second part of the algorithm, leading to a total time complexity of $O(n * logn)$.

The StdB. method has another input that affects the time complexity – the size of the sub-signal – we denote it as m. Dividing the signal into sub-signals will take $O(n)$, and calculating the standard deviation will take $O(m)$. Note that we can optimize this part due to the fact that each sub-signal's standard deviation is based on the standard deviation of the previous sub-signals, but we do not do so here, because the overall time complexity will not benefit from it. The total time complexity so far is $O(n * m)$. Next, we perform the following steps until the output is the desired size: We find the lowest standard deviation, and for each of the points in the sub-signal with this standard deviation, we see if it can be dropped by checking whether it appears in other sub-sections we have not checked yet. Iterating until we drop enough point adds $O(n)$, checking each point in the sub-signal adds $O(m)$ and iterating over the sub-signals again is $O(m)$. Finding the lowest standard deviation does not add to the overall time complexity. Therefore, the total time complexity of the algorithm is $O(n^2 * m)$.

The results of our theoretical analysis show that the advantages of our proposed methods demonstrated in the previous experiments come at the expense of being slower than the SOTA methods. Further, we can see that all of the methods that evaluate a single point have the same complexity, while the algorithm that evaluates a sub-signal has a greater time complexity.

IX. Discussion and Conclusions

In this paper, we proposed various UTSD downsampling methods that were comparable to commonly used methods in terms of their output's similarity to the input and their generalization capabilities in various learning tasks (i.e., classification, regression, and next point prediction) as well as compared the methods through theoretical time complexity analysis. We also performed a comprehensive evaluation to examine the proposed methods and metrics. We conclude that it is always best to use the Iterative Method if the aim is to preserve the original signal, but if the aim is to maintain the model's generalization capabilities, there is no single best method for all types of tasks and datasets. Our evaluation, using the proposed and existing metrics, demonstrates our metrics' suitability for assessing downsampling methods and their ability to serve as reasonable replacements for the existing metrics.

Acknowledgment

We would like to thank Ben-Gurion University's Data Science Research Center and the Cyber Security Research Center for their support.

References

[1] G. Yang, X. Chen, T. Zhang, S. Wang, and Y. Yang, "An Impact Study of Concept Drift in Federated Learning," in Proceedings - IEEE International Conference on Data Mining, ICDM, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1457–1462. doi: 10.1109/ICDM58522.2023.00191.

[2] S. Steinarsson, "Downsampling Time Series for Visual Representation," University of Iceland, 2013. [Online]. Available: https://skemman.is/bitstream/1946/15343/3/SS_MSthesis.pdf

[3] S. Rabanser, T. Januschowski, V. Flunkert, D. Salinas, and J. Gasthaus, "The effectiveness of discretization in forecasting: An empirical study on neural time series models," arXiv preprint arXiv:2005.10111, 2020.

[4] O. Landau, A. Cohen, S. Gordon, and N. Nissim, "Mind your privacy: Privacy leakage through BCI applications using machine learning methods," Knowl Based Syst, vol. 198, p. 105932, 2020, doi: 10.1016/j.knosys.2020.105932.

[5] C. Chia, "Using Adaptive Downsampling to Compare Time Series with Warping," 2010, doi: 10.1109/ICDMW.2010.94.

[6] Y. Wei et al., "Compatible Transformer for Irregularly Sampled Multivariate Time Series," in Proceedings - IEEE International Conference on Data Mining, ICDM, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1409–1414. doi: 10.1109/ICDM58522.2023.00183.

[7] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances," Data Min Knowl Discov, vol. 31, pp. 606–660, 2017.

[8] Haldun Akoglu, "User's guide to correlation coefficients," Turk J Emerg Med, vol. 18, no. 3, pp. 91–93, 2018.

[9] Berkeley Seismological Laboratory, "The northern california earthquake data center."

[10] L. Ye, "Time Series Shapelets : A New Primitive for Data Mining," in ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2009.

[11] J. Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," in Proceedings of the 22nd ACM international conference on Multimedia, 2014, pp. 1041–1044.

[12] P. PhysioBank, "Physionet: components of a new research resource for complex physiologic signals," Circulation, vol. 101, no. 23, pp. e215–e220, 2000.

[13] H. Fanaee-T, "Bike Sharing," 2013.

[14]https://archive.ics.uci.edu/dataset/409/daily+demand+forecasting+orders

[15]https://www.kaggle.com/datasets/daiearth22/uniqlo-fastretailing-stock-price-prediction

[16]https://www.kaggle.com/datasets/sumanthvrao/daily-climate-time-series-data?resource=download

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:58:31 UTC from IEEE Xplore. Restrictions apply.